

Efficient Binary Trees

Additional Exercises related to Lecture 4
Complexity and Algorithms

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



Question 1

Consider the following data set:

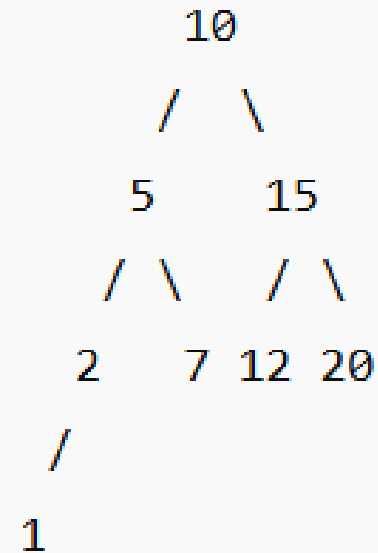
10, 5, 2, 1, 7, 15, 12, 20

1. Create an Un-balanced Binary Search Tree
2. Is created tree balanced or not, if not, what is the reason (S) for the violation
3. Transforming a binary search tree into a backbone (vine)
4. Perform global tree balancing using the DSW algorithm (Day-Stout-Warren) on the vine (backbone) created in step 2 above.
5. Write a pseudocode algorithm for single and double rotations in a general case

Answer 1: Step-by-Step BST Insertion

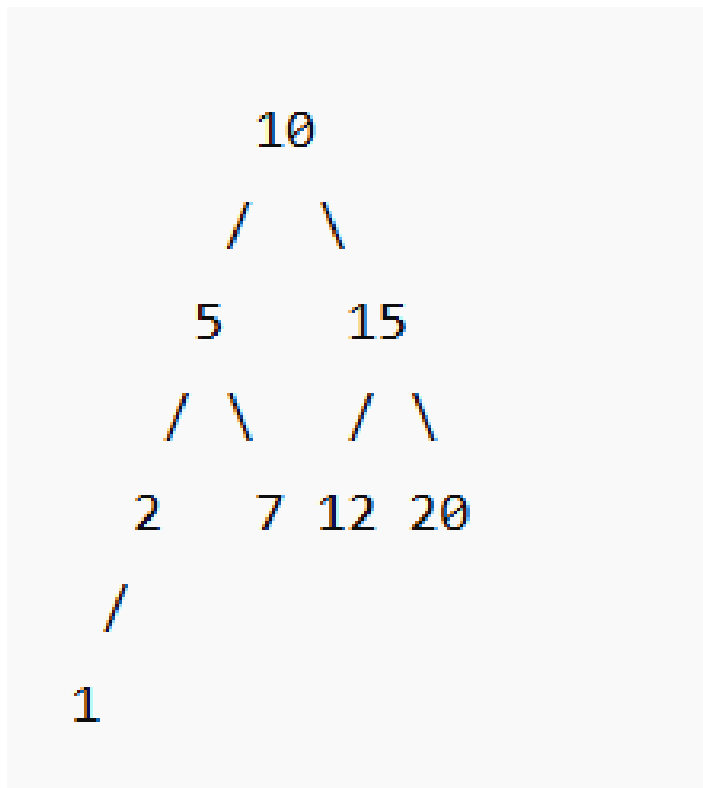
To build a BST from the list 10, 5, 2, 1, 7, 15, 12, 20, we insert elements one by one:

1. **Insert 10** → Root node.
2. **Insert 5** → $5 < 10$ → goes to the left of 10.
3. **Insert 2** → $2 < 10$, $2 < 5$ → goes to the left of 5.
4. **Insert 1** → $1 < 10$, $1 < 5$, $1 < 2$ → goes to the left of 2.
5. **Insert 7** → $7 < 10$, $7 > 5$ → goes to the right of 5.
6. **Insert 15** → $15 > 10$ → goes to the right of 10.
7. **Insert 12** → $12 > 10$, $12 < 15$ → goes to the left of 15.
8. **Insert 20** → $20 > 10$, $20 > 15$ → goes to the right of 15.



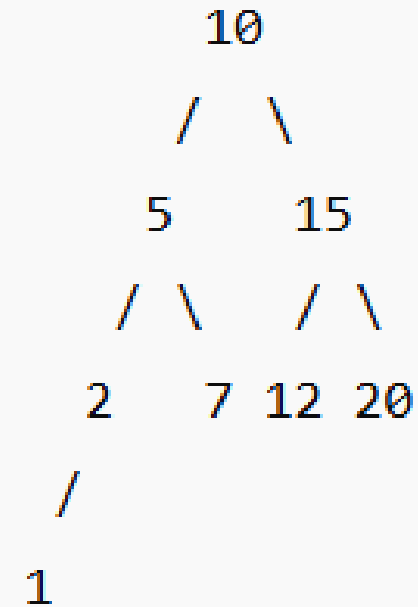
Answer

- Data set: 10, 5, 2, 1, 7, 15, 12, 20
- Unbalanced BST



Answer

- 2.
- This is **not balanced**
- The left subtree of 5 is deeper than the right.



Answer 3: Transform the BST into a **Backbone (Vine)**

- Step : Transform the BST into a Backbone (Vine)
- Goal: Transform the BST into a **right-skewed structure** (like a linked list) using **right rotations**, each node should only have a right child.
- Step-by-Step Rotations:

Let's denote:

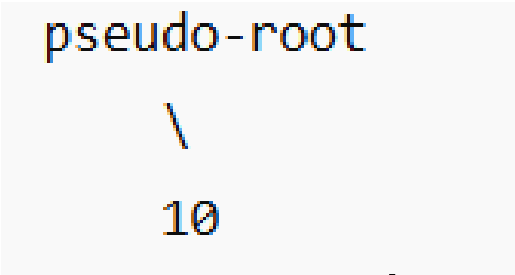
P = parent node

C = current node

Answer 3: Transform the BST into a **Backbone (Vine)**

- Backbone (Vine):
- A **vine** is a right-skewed linked list where every node only has a right child.
- We'll use **right rotations** to eliminate all left children.

- Intermediate Steps:
- Start with pseudo-root:



```
graph TD; pseudo-root --> 10;
```

The diagram shows a node labeled "pseudo-root" with a right-pointing arrow leading to a node containing the value "10".

Now traverse down, performing **right rotations** where left child exists.

Answer 3: Transform the BST into a **Backbone (Vine)**

- Start with a **pseudo-root** pointing to 10
- Step 1: Rotate at 10 (left child =
- Original Tree

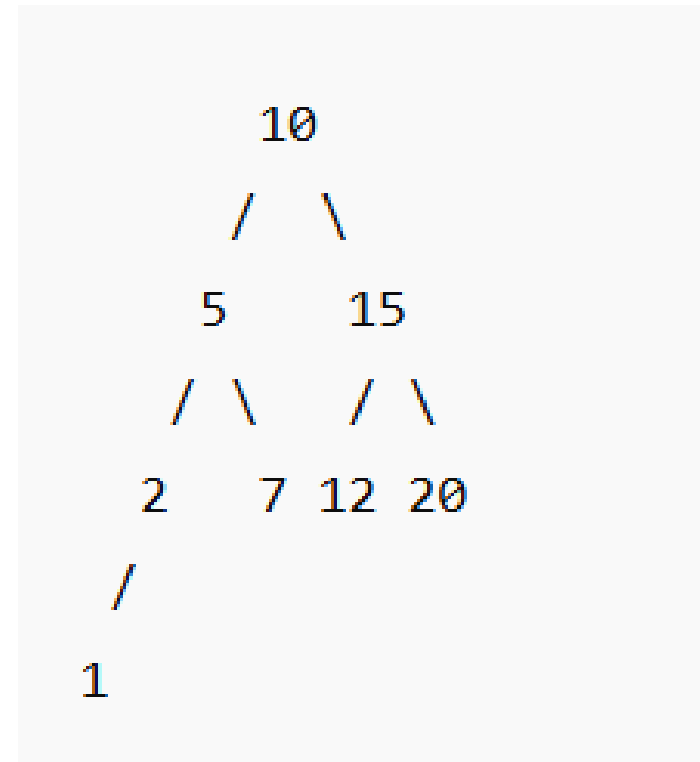
Before:

pseudo-root

\
10
/
5

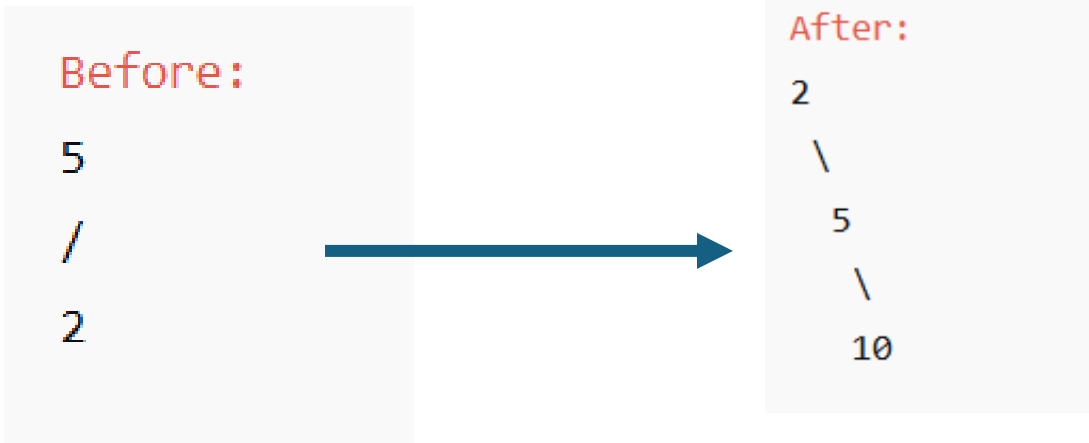
After right rotation at 10:
pseudo-root

\
5
\
10

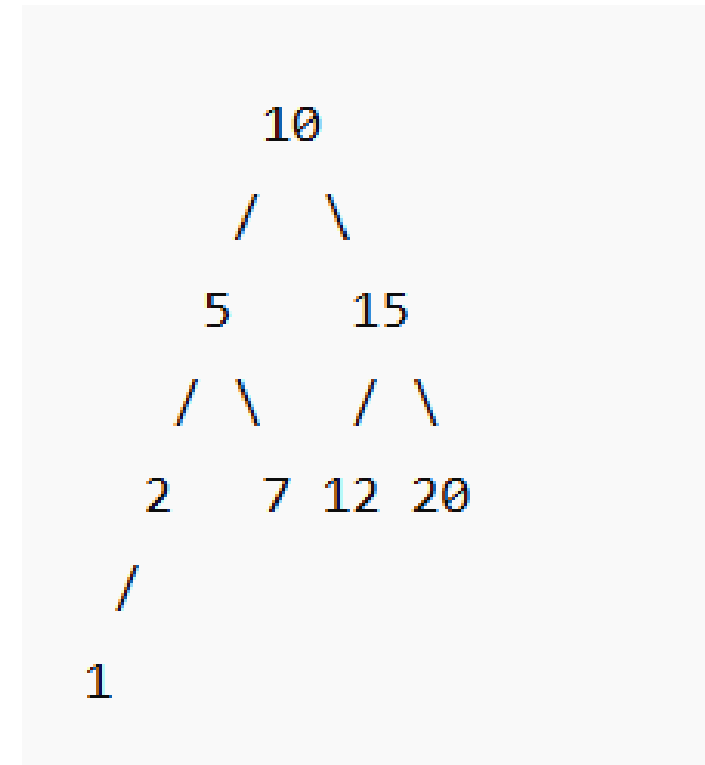


Answer 3: Transform the BST into a **Backbone** (**Vine**) Original Tree

- Step 2: Rotate at 5 (left child = 2)

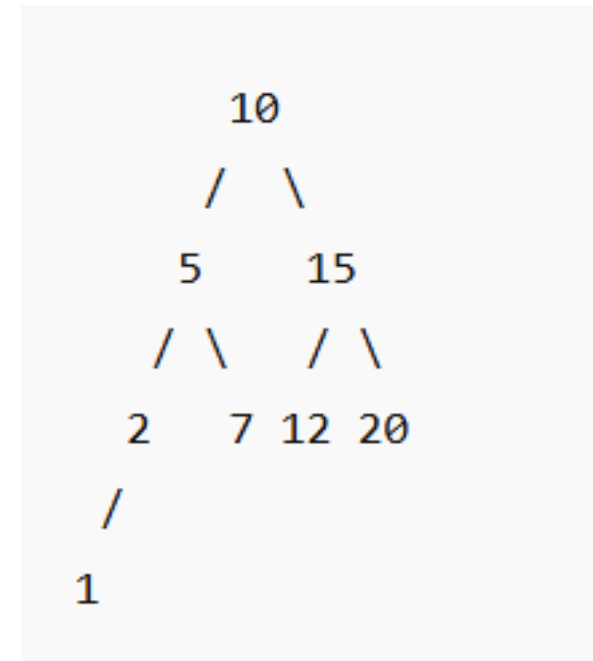
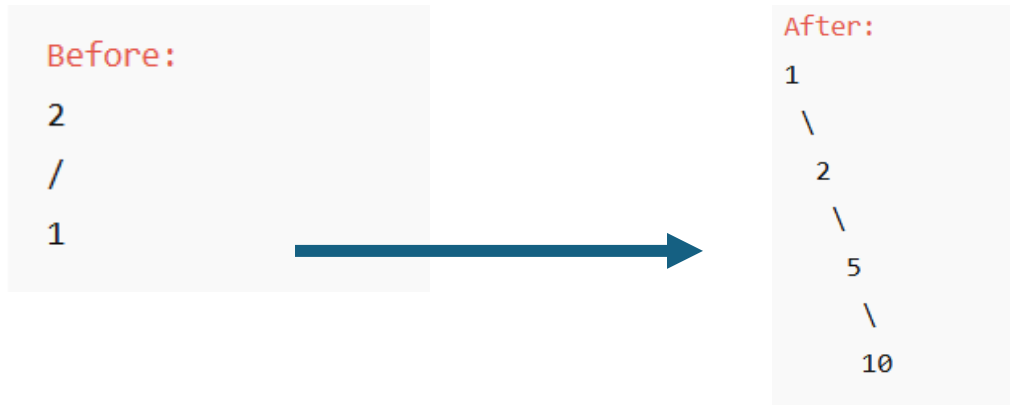


Original Tree

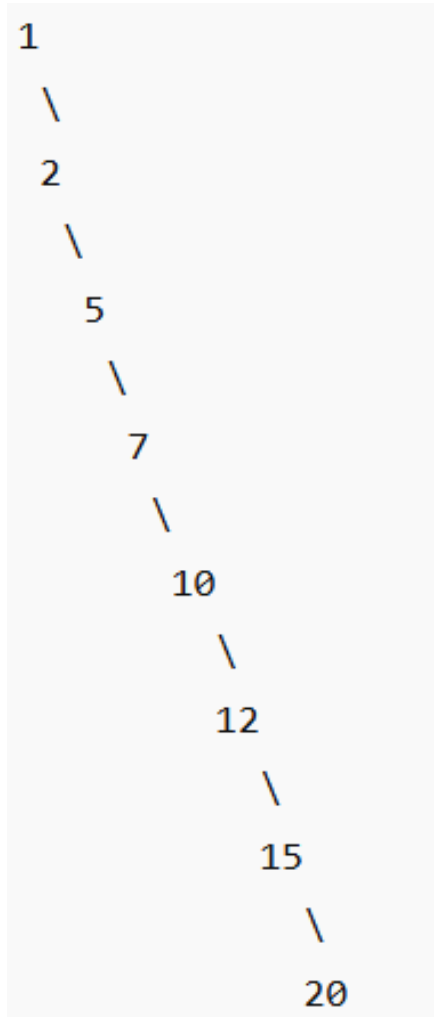


Answer 3: Transform the BST into a **Backbone (Vine)**

- Step 3: Rotate at 2 (left child = 1)
- Original Tree



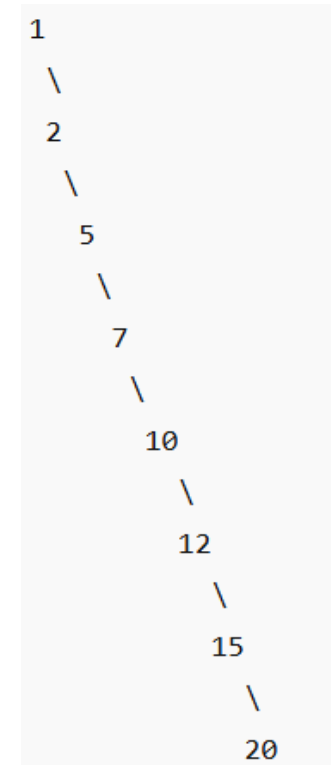
Answer 3: Transform the BST into a **Backbone (Vine)**



- Each node has **only a right child** – this is the **backbone or vine**.

Answer 4: Perform global tree balancing using the DSW algorithm

- Step-by-Step: DSW Global Tree Balancing
- Backbone (Vine)
- Refer slide no 18 in Lecture 4



Question 2 : Answers to the question

1. Create an AVL tree using the following dataset:

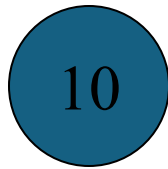
10, 20, 30, 25, 27, 7, 4, 23, 26, 21.

You may show the step-by-step procedure, specifying the type of rotation involved at each intermediate step.

2, Write pseudocode algorithms or C code to implement an AVL tree, including single and double rotations for the general case.

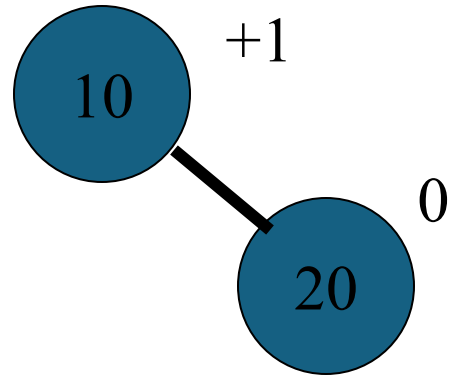
Construction of an AVL trees.

- Eg. 10,20,30,25,27,7,4,23,26,21
- We begin by inserting integer 10 into the root

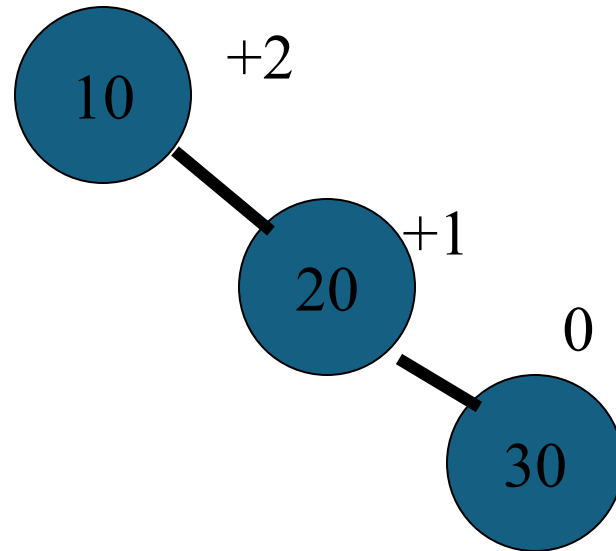


- This node satisfies the AVL condition.

Now we add another node 20



Now we add third node, 30



The root node 10 has a difference of 2, which violates the AVL conditions.

Therefore, we must re-arrange the node to restore the balance in the tree.

We perform an operation known as the rotation when the tree goes out of balance. There are two types of rotation used in AVL trees. **Single rotation and Double rotation.**

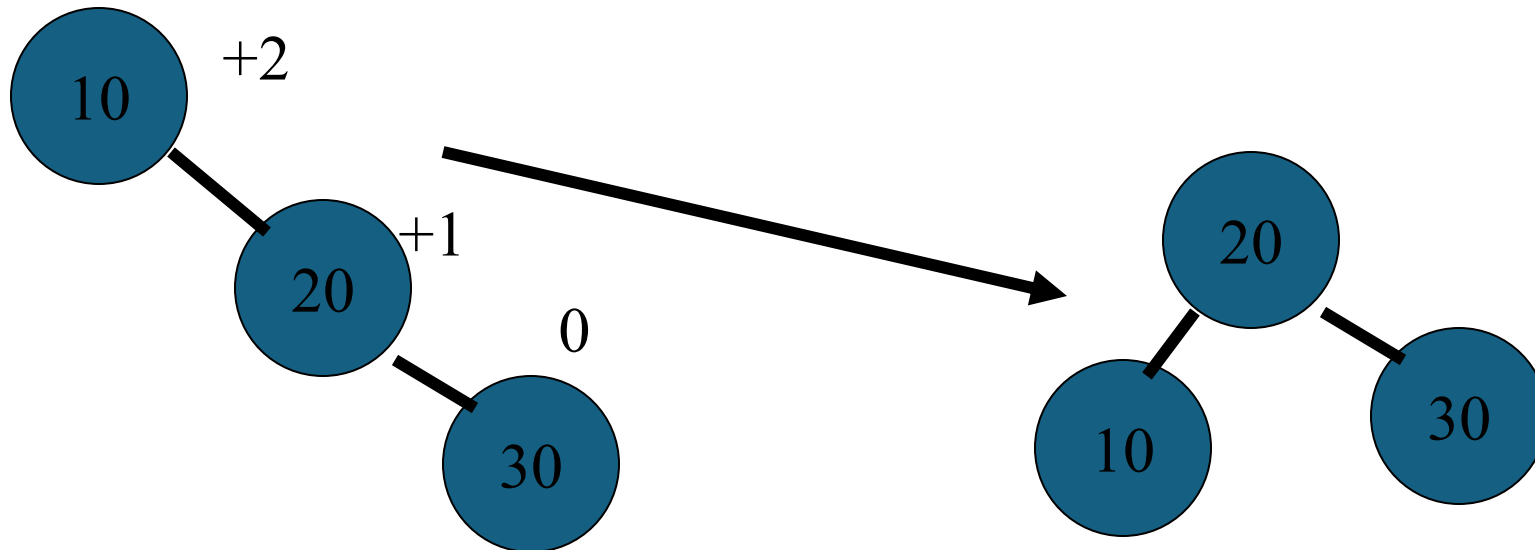
The rules for deciding which type of rotations to be used.

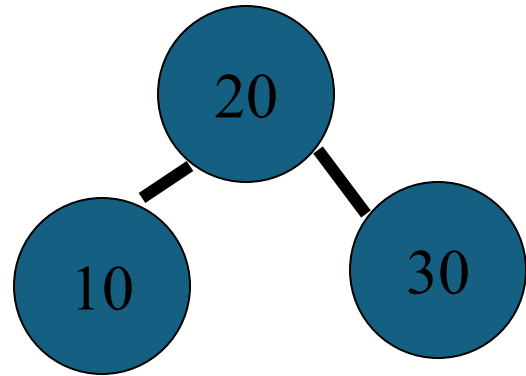
1. When you have found the first node is out of balance, restrict your attention to that node and the two nodes in the two layers immediately below it.
2. If these three nodes lie in a straight line, single rotation is needed to restore the balance.
3. If these three nodes lie in a “dog-leg” pattern You need double rotation to restore the balance.

In our example. Node 10 is imbalanced, consider two layers immediately below this node. These nodes lie in a straight line. So we need a single rotation to restore the balance

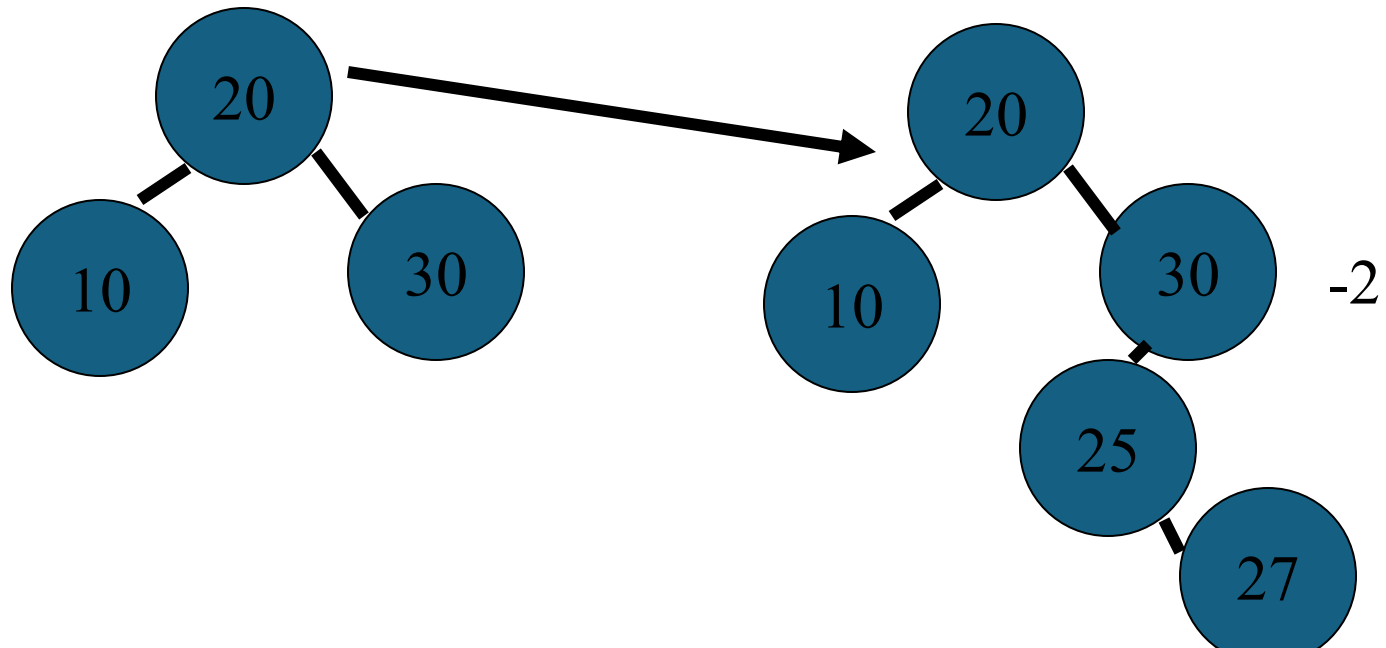
Single Rotation :

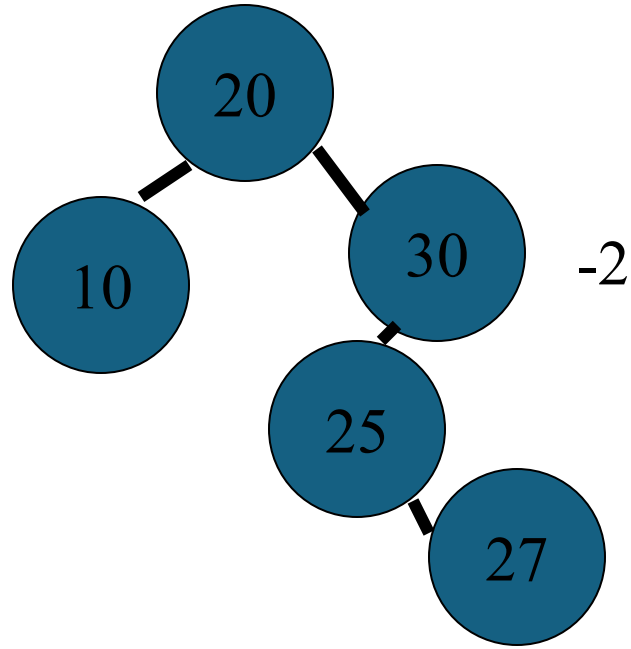
- This involves shifting the middle node up to replace the top node down to become the left child of the middle.





We continue by adding two more nodes 25 & 27



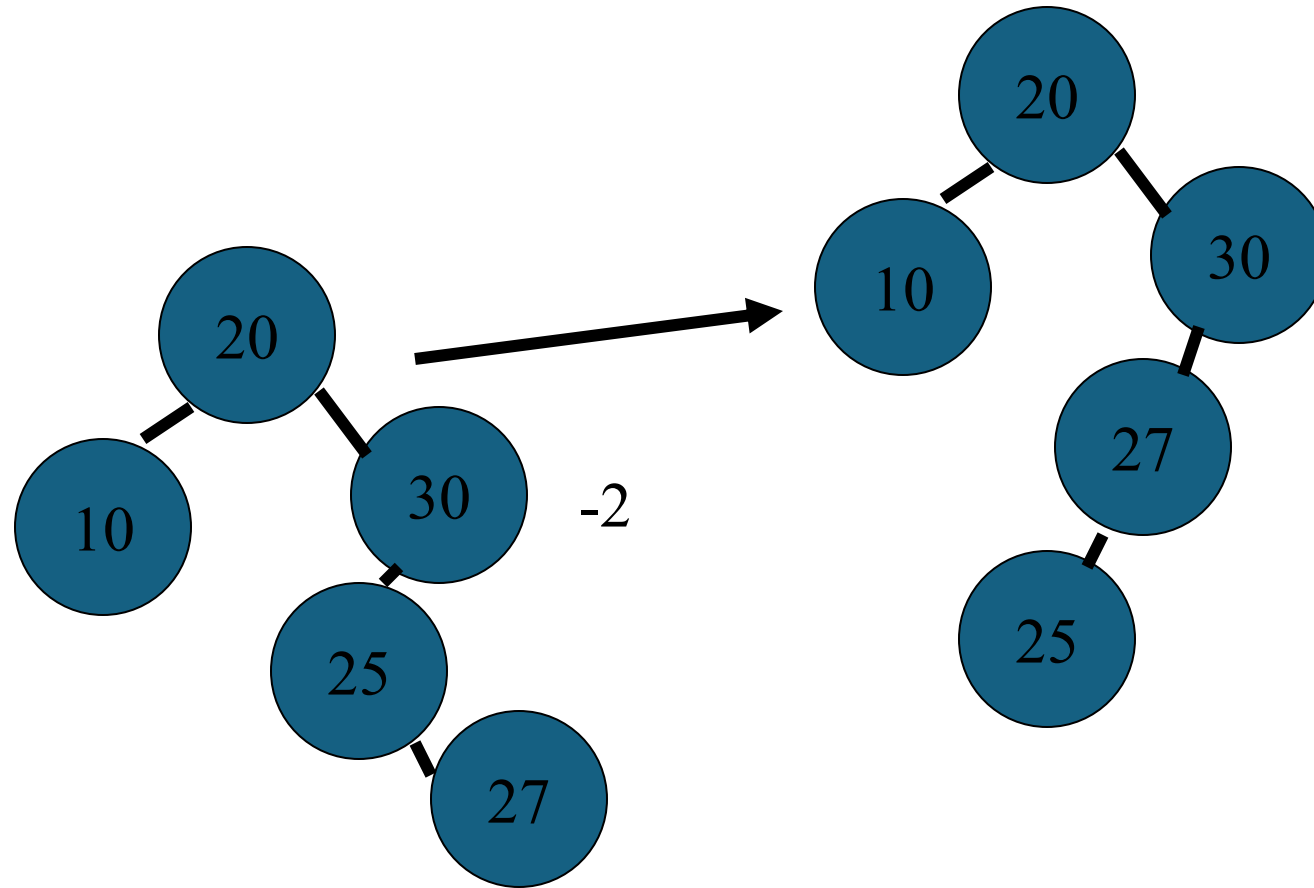


The first imbalance is detected at node 30 these three nodes form a dog-leg pattern.

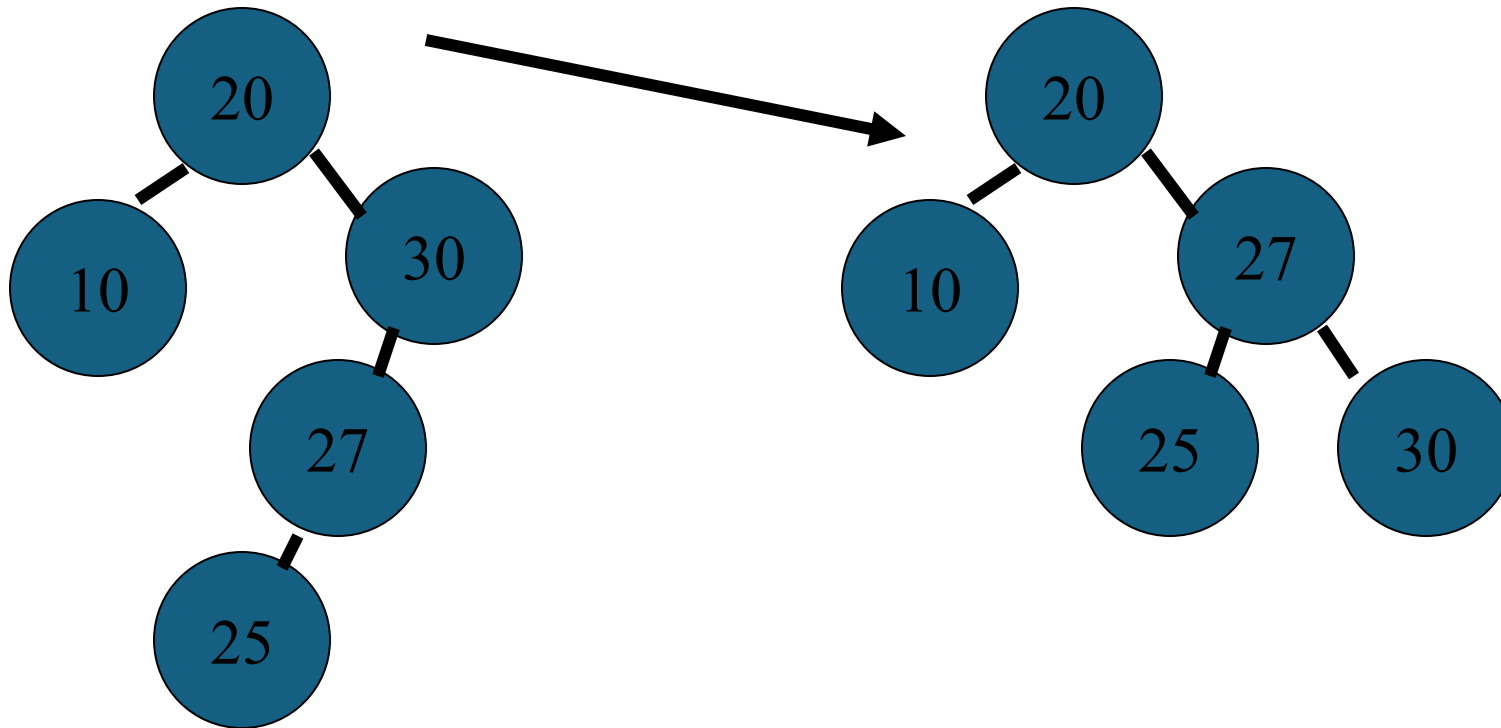
We require a double rotation to correct the balance.

Double rotation consists of two single rotations. These two rotations are in opposite directions.

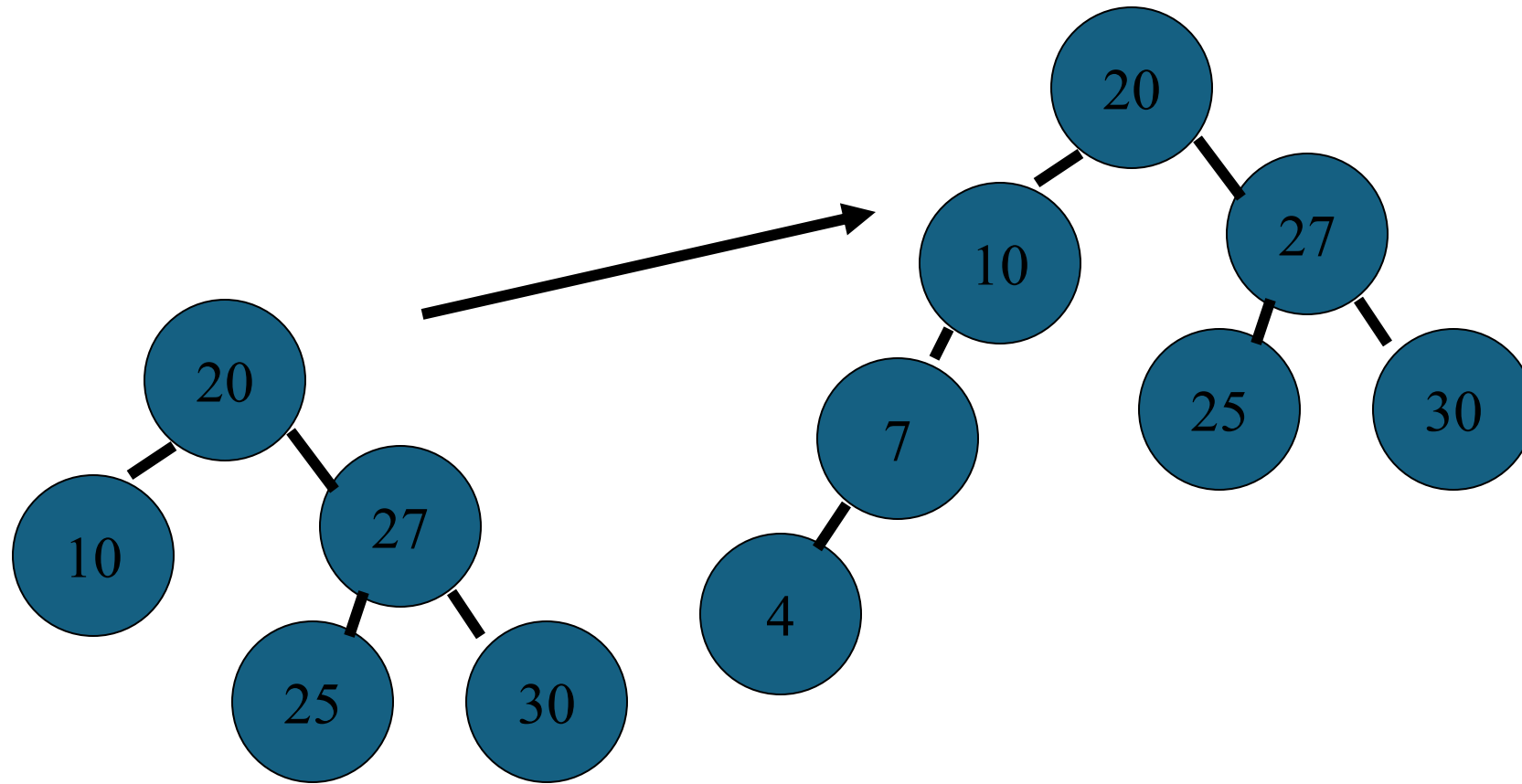
1st Rotation: we rotate the node 27 up to 25, and 25 down to become the left child of 27.



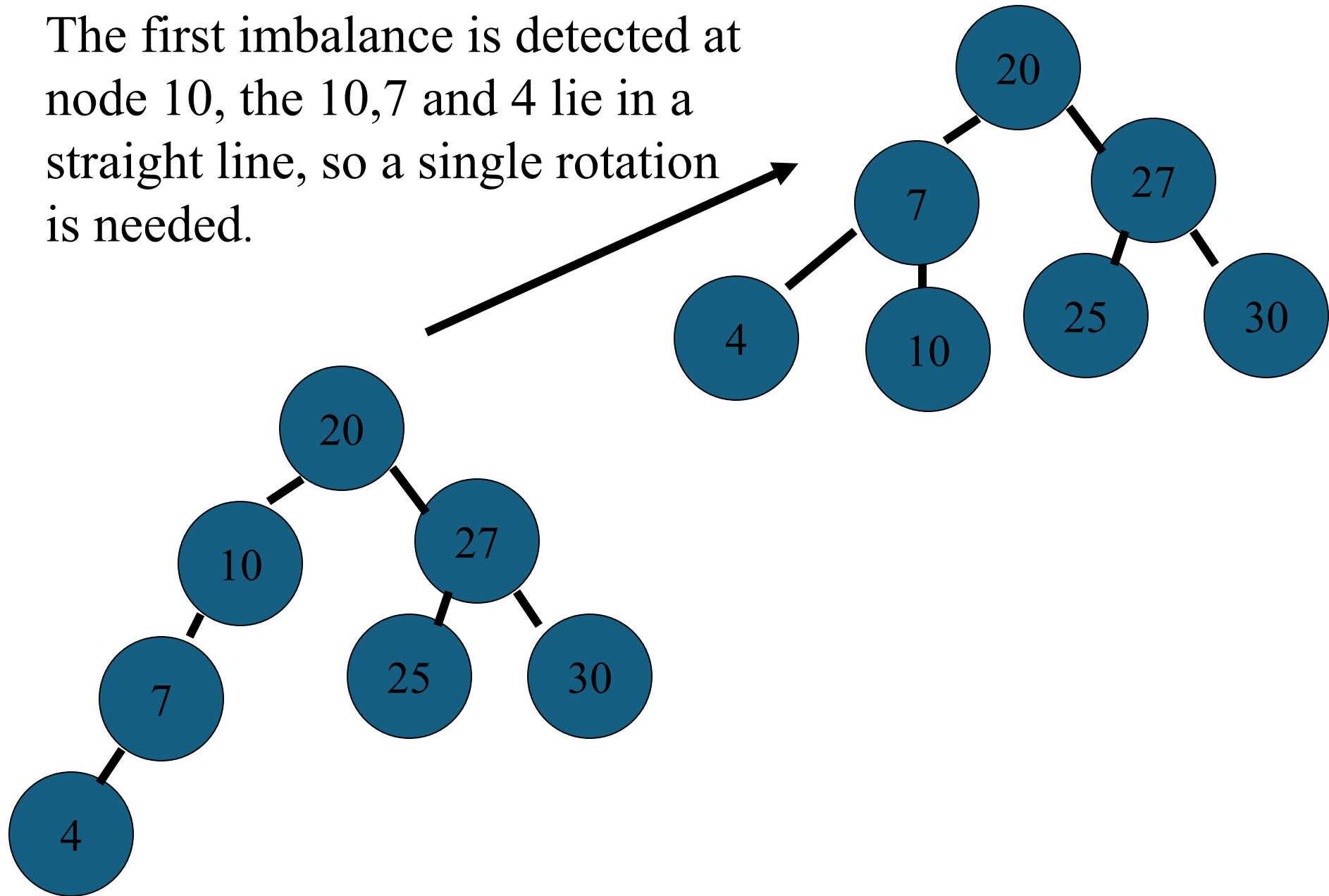
2nd Rotation : Node 27 rotates up to replace 30 and node 30 rotates down to become the left child of 27



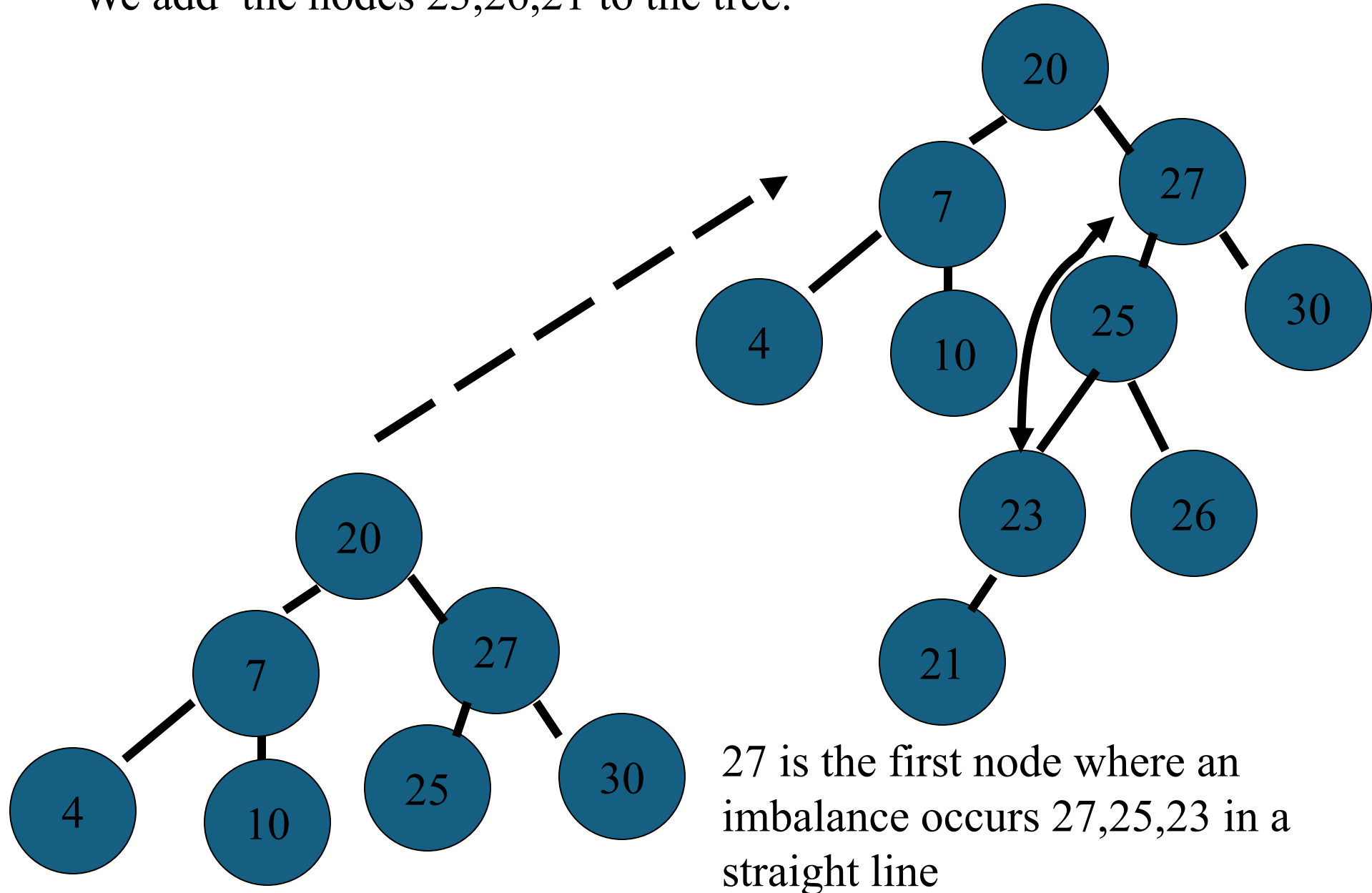
We continue by adding the nodes 7 and 4 to the tree.



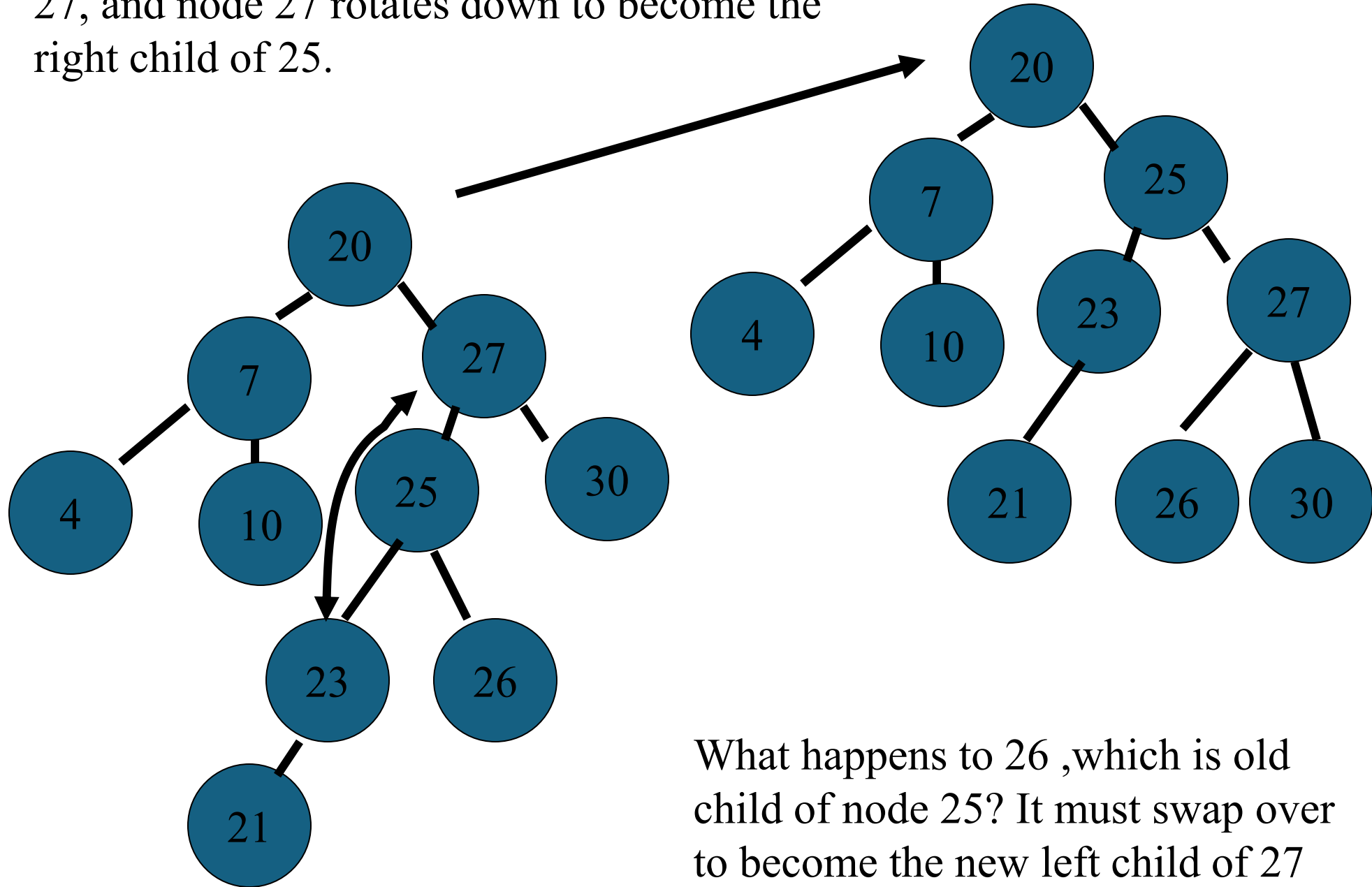
The first imbalance is detected at node 10, the 10,7 and 4 lie in a straight line, so a single rotation is needed.



We add the nodes 23,26,21 to the tree.

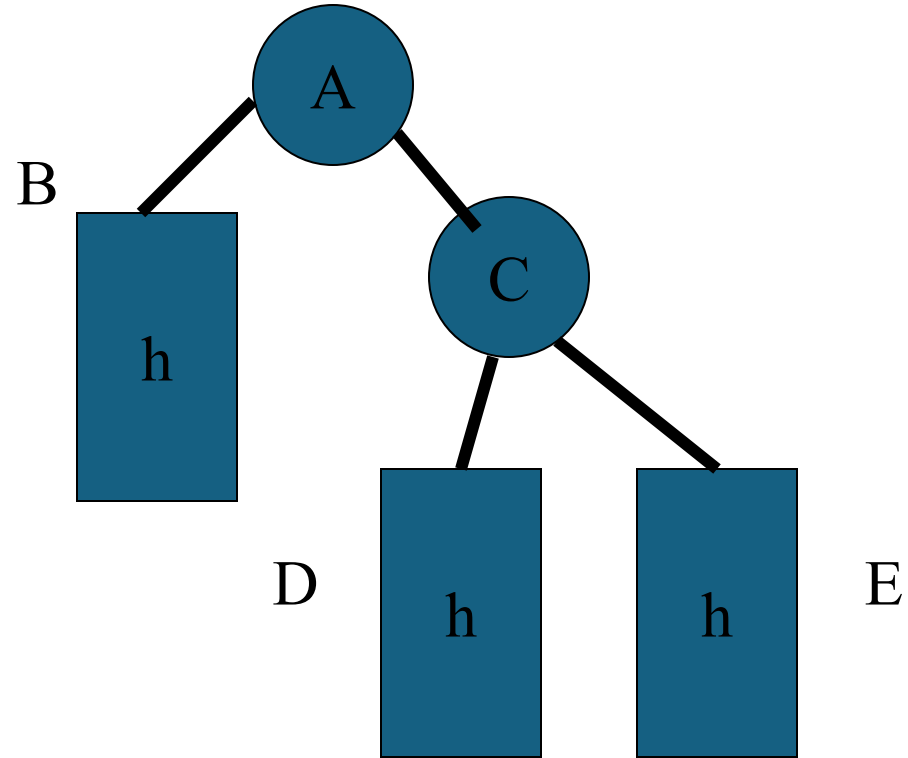


The middle node (25) rotate up to replace node 27, and node 27 rotates down to become the right child of 25.



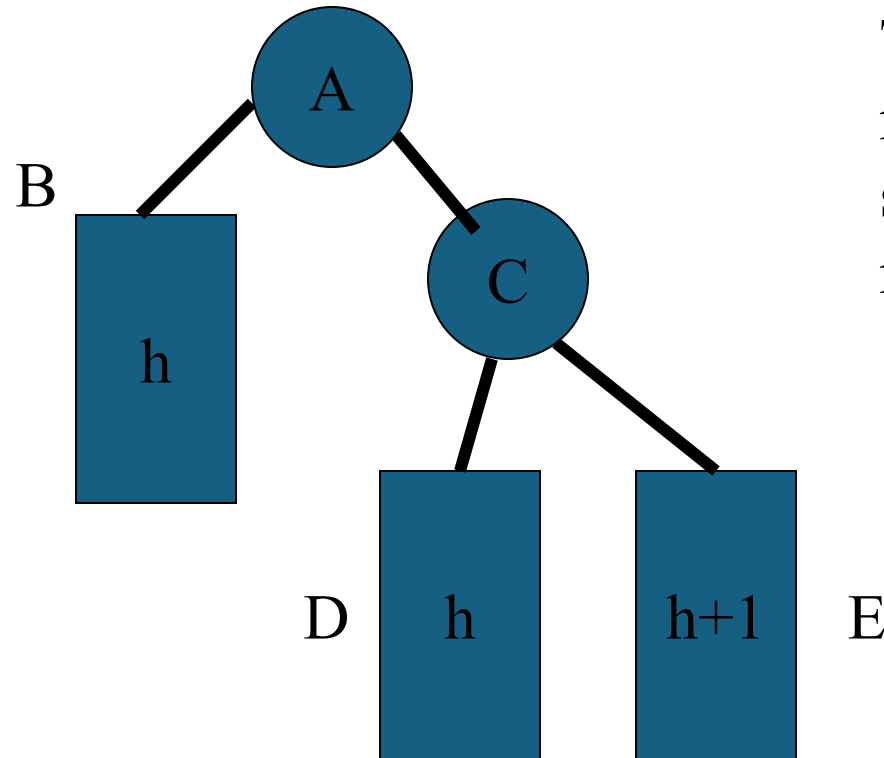
What happens to 26 ,which is old child of node 25? It must swap over to become the new left child of 27

Single Rotation



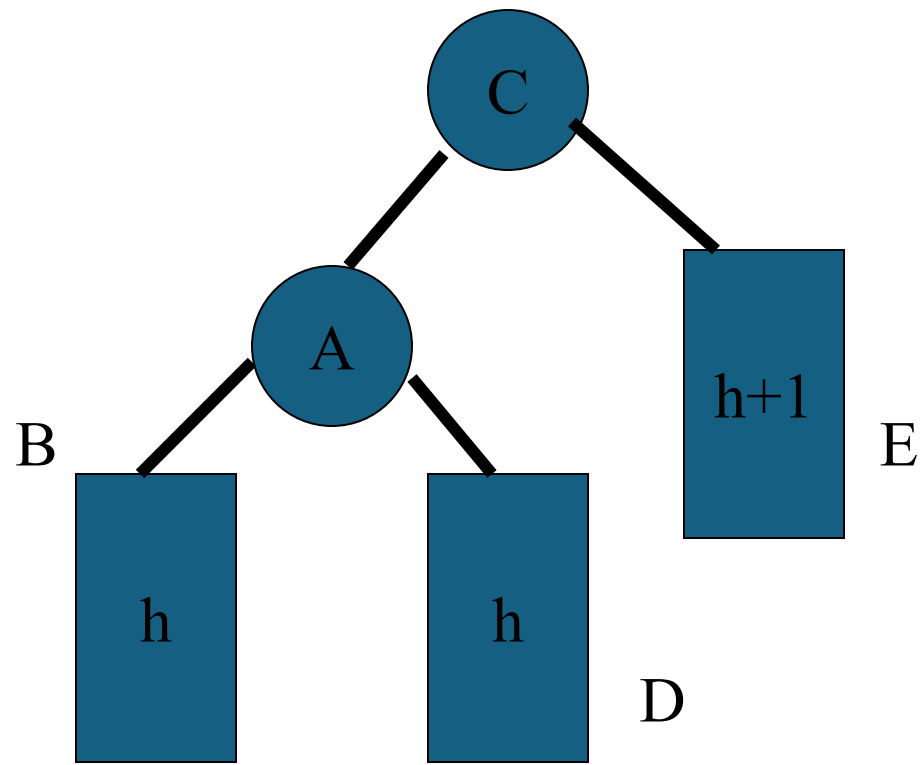
Suppose we want to insert a node into the subtree under node E

The tree is imbalance at node A



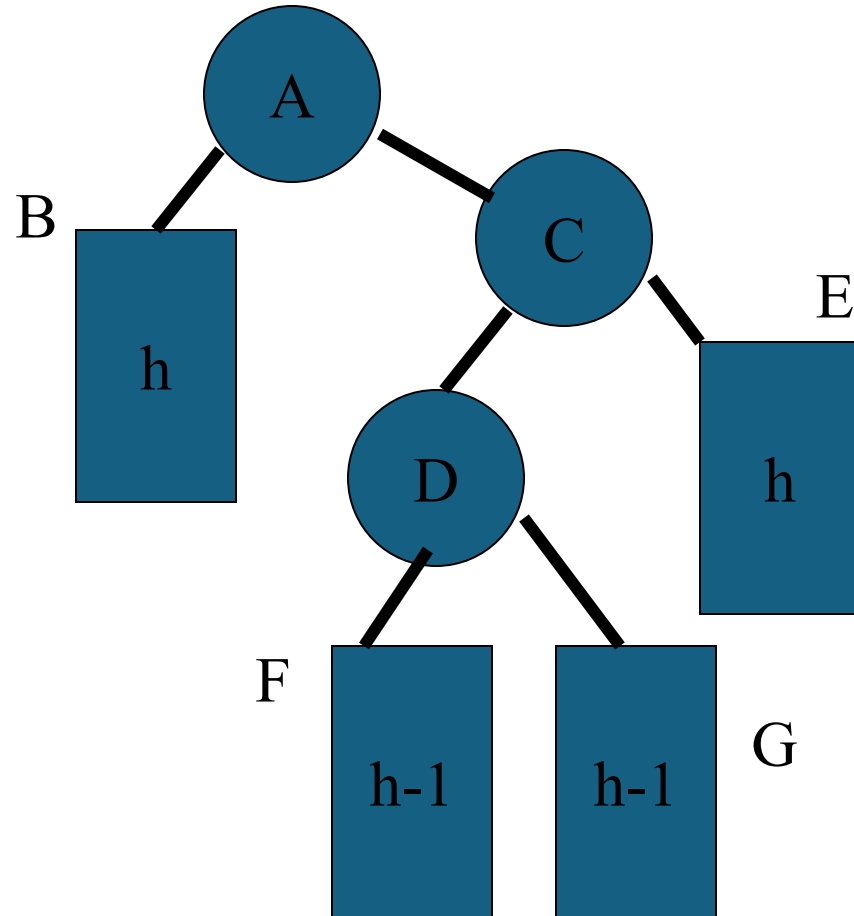
The two nodes immediately below node A. Nodes A,C,E are in the straight line. So we need a single rotation.

We rotate C up to replace A and A down become the left child of C. Node D must swap over to become the new child of A



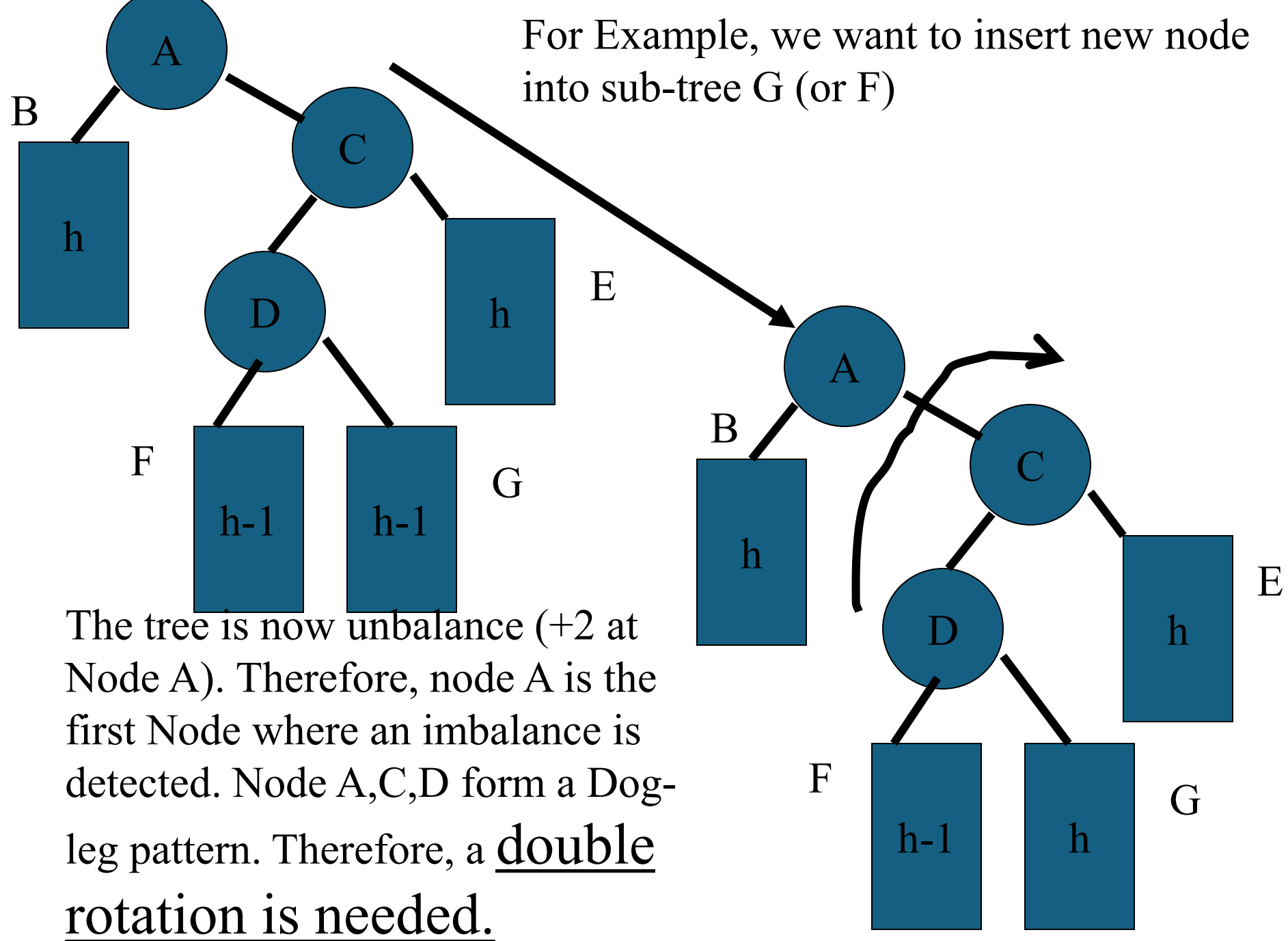
This tree is balanced.

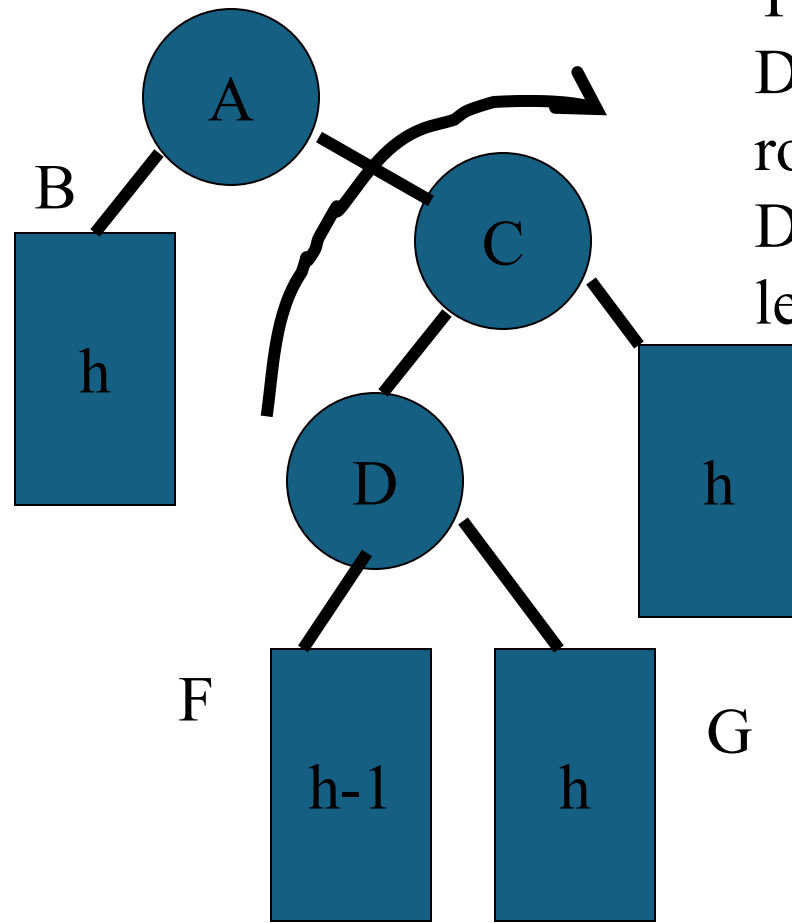
Double Rotation



This tree is an AVL tree,
Assuming all sub-trees in the
rectangular boxes are AVL
trees. The height differences
are 0 at node D & C and +1 at
Node A.

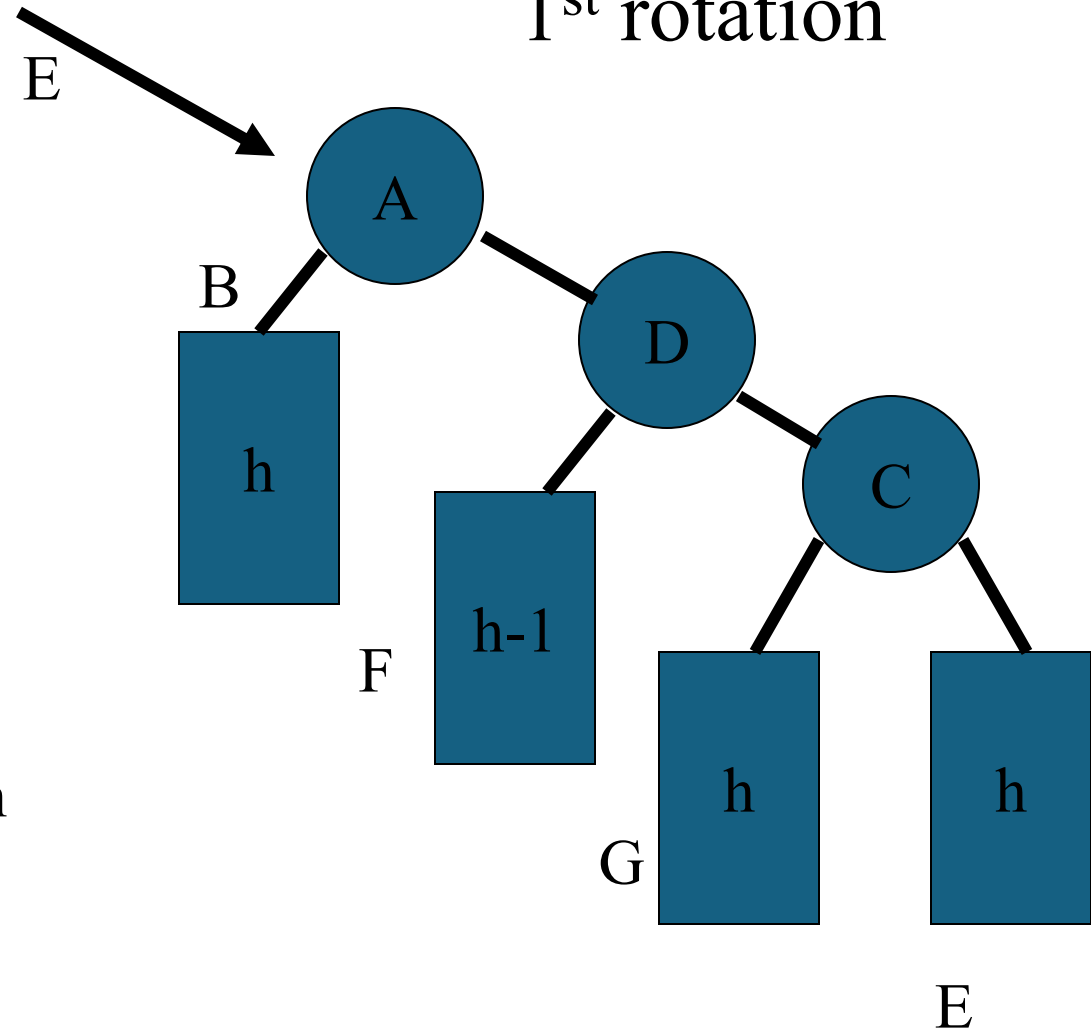
For Example, we want to insert
new node into sub-tree G (or F)



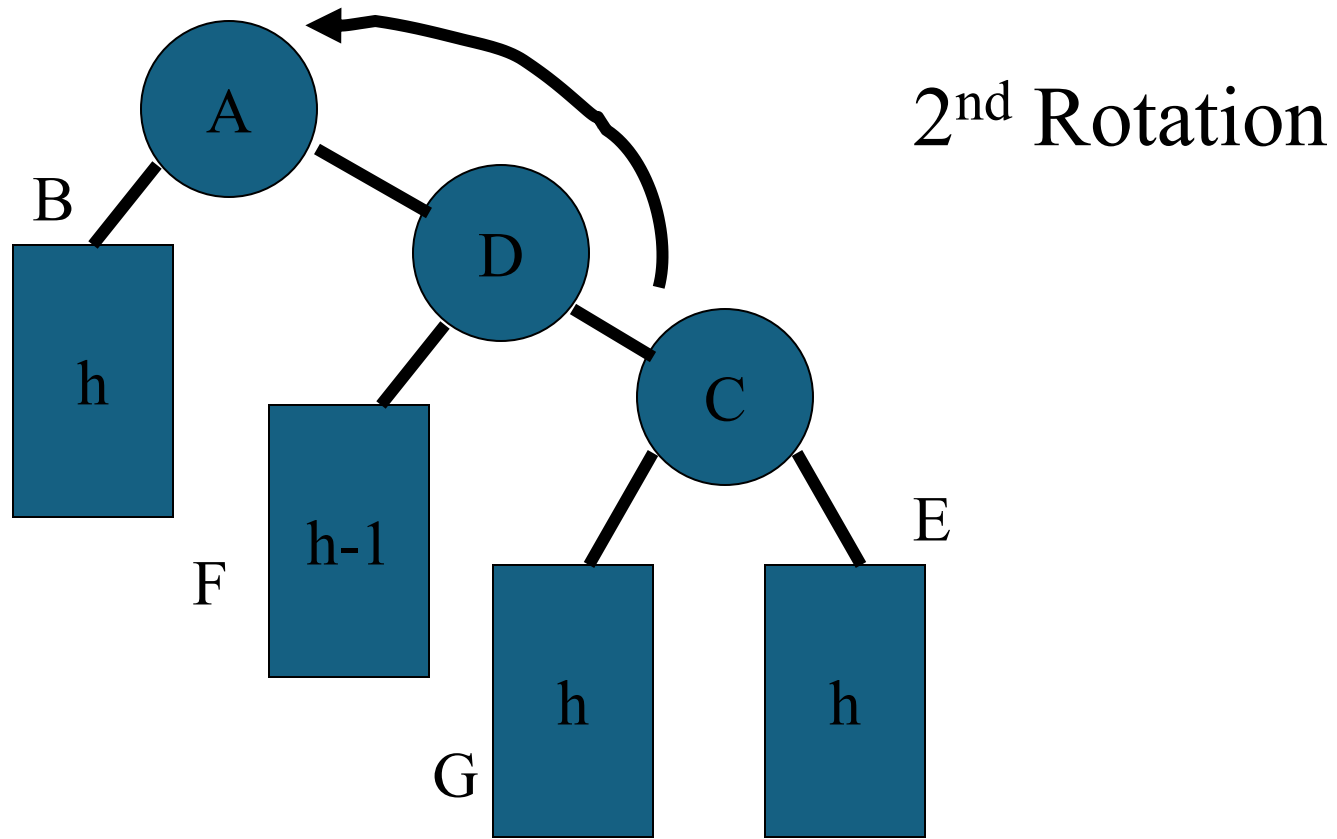


The first rotation involves node C and Node D: Node D rotates up to replace C and C rotates down to become new right child of D. Sub-tree G swaps over to become the left child of node C.

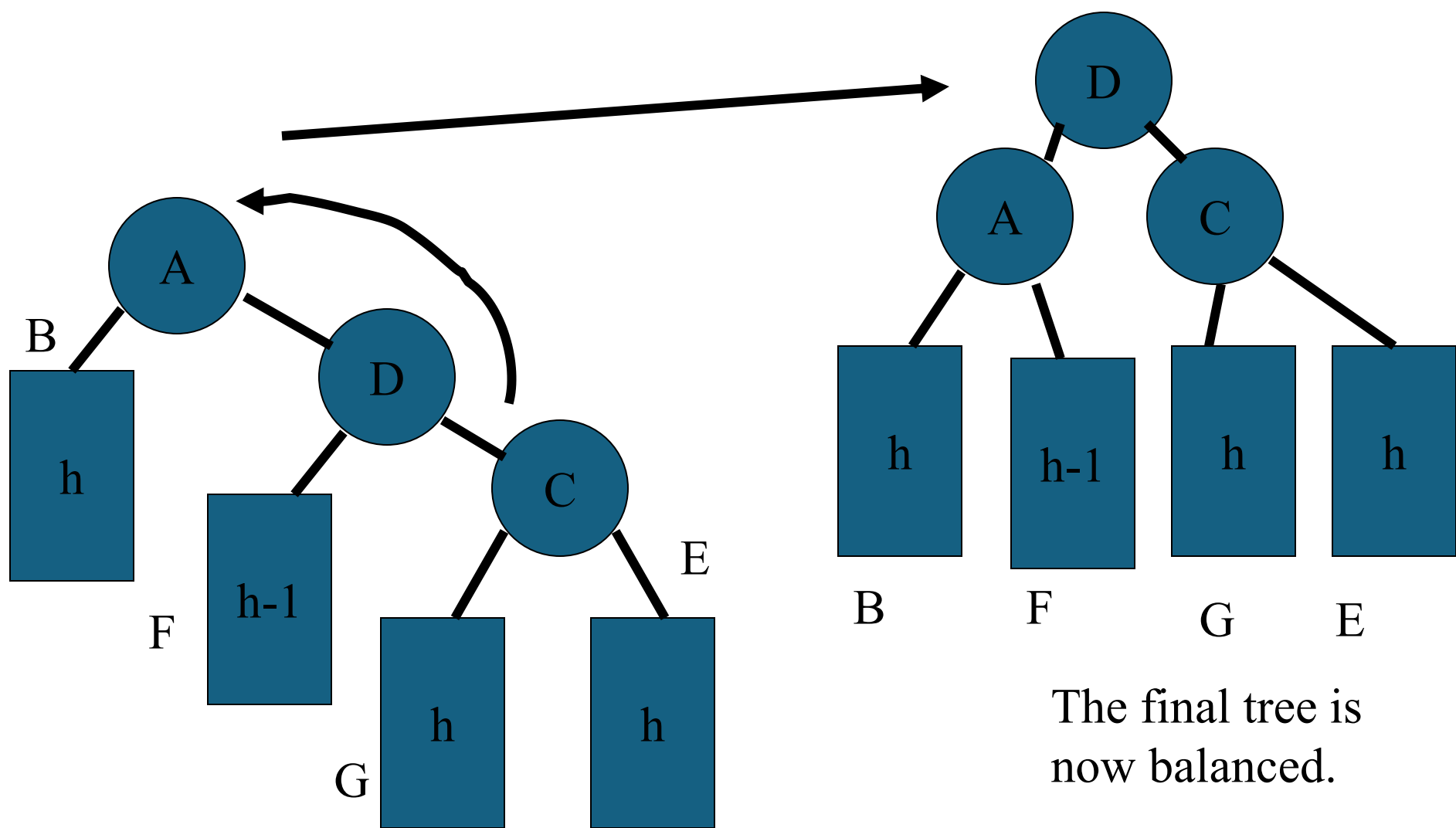
1st rotation



This first rotation does not solve the imbalance problem since the height differences are still $+2$ at both nodes A and D.



Therefore, we perform the 2nd rotation, which is in the opposite direction to the first one, and which involves nodes A,D and C



Node D rotates up to replace A. Node A rotates down to become the left child of D, and Sub-tree F swaps over to become the new right child of node A.

Answer 5: Write a pseudocode algorithm for single and double rotations in a general case

- Refer page nos. (144-157), Fourth Edition Data Structures and Algorithm Analysis in C++, Mark Allen Weiss Florida International University,